

Bitonic Sort als Sortieralgorithmus in parallellisierten Anwendungen

Berend Tobias
Angewandte Informatik
Hochschule Offenburg
Offenburg, Deutschland
tberend@stud.hs-offenburg.de

I. EINLEITUNG

- Kurze Aussicht darauf, warum Bitonic Sort effektiv ist als Sortieralgorithmus
- Begriffsdefinition „Bitonic“
- Aussicht auf Komplexität von $O(n \log(n)^2)$ sowie $O(\log(n)^2)$

II. PARALLELE PROZESSE

- Ganz kurz “Was ist Parallelität“ und warum Vorteilhaft
- Beispiele von parallelen Anwendungen
 - o Threads
 - o Prozesse
 - o CPUs

III. FUNKTIONSWEISE BITONIC SORT

- Bitonic Sort ruft Bitonic Sort (Rekursiv) und Bitonic Merge auf
- Vorbedingung: Arraylänge $l = 2^x, x \in \mathbb{N}_0$

III.A. Bitonic Sort

1. Array wird rekursiv in zwei Teilarrays aufgeteilt
2. Das erste wird aufsteigend Sortiert, das zweite absteigend (durch Bitonic Merge)
3. Bricht Rekursion bei $l \leq 1$ ab

III.B. Bitonic Merge

Sortiert die Teilarrays aufsteigend bzw. absteigend, sowie das ganze Array durch Tauschen der Arrayelemente und anschließendes rekursives Aufrufen.

1. Tauscht Element E_i mit E_{i+n} , $n = \frac{l}{2}$, $i \in [0; n)$ falls $E_i < E_{i+n}$ und aufsteigend sortiert werden soll oder falls $E_i > E_{i+n}$ und absteigend sortiert werden soll.
2. Ruft sich rekursiv für zwei Teilarrays auf
3. Bricht Rekursion bei $l \leq 1$ ab

IV. LAUFZEITOPTIMIERUNG DURCH PARALLELISIERUNG

IV.A. Lineare Implementierung

- Komplexitätsklasse $O(n \log(n)^2)$
- Erklärung durch $\log(n)$ Rekursionsstufen und n Vergleiche über $\log(n)$ Stufen

IV.B. Parallele Implementierung

- Komplexitätsklasse $O(\log(n)^2)$
- Erklärung durch $\log(n)$ Rekursionsstufen und n Vergleiche über $\log(n)$ Stufen, jedoch mit n Threads gleichzeitig

IV.C. Benchmarks

Vergleich von linearer & paralleler Implementierung sowie anderer Sortieralgorithmen (Tabelle oder Abbildung)

V. VERGLEICH MIT ANDEREN SORTIERALGORITHMEN

- Vergleich von Bitonic Sort mit anderen Sortieralgorithmen:
 - o Komplexität
 - o Voraussetzung
 - o Implementierung

VI. FAZIT

- Vorteile Bitonic Sort
- Nachteile Bitonic Sort
- Welche Anwendungen nutzen Bitonic Sort sinnvoll

LITERATUR

- [1] Z. Yildiz, M. Aydin and G. Yilmaz, "Parallelization of bitonic sort and radix sort algorithms on many core GPUs," *2013 International Conference on Electronics, Computer and Computation (ICECCO)*, Ankara, Turkey, 2013, pp. 326-329, doi: 10.1109/ICECCO.2013.6718294.
- [2] K. Velusamy, T. B. Rolinger, J. McMahon and T. A. Simon, "Exploring Parallel Bitonic Sort on a Migratory Thread Architecture," *2018 IEEE High Performance Extreme Computing Conference (HPEC)*, Waltham, MA, USA, 2018, pp. 1-7, doi: 10.1109/HPEC.2018.8547568.
- [3] Jae-Dong Lee, Kyung-Hee Kwon and Young-Beom Park, "Minimizing communication in bitonic sorting software," *Proceedings 1997 International Conference on Parallel and Distributed Systems*, Seoul, Korea (South), 1997, pp. 166-171, doi: 10.1109/ICPADS.1997.652545.
- [4] B. M. Gocal and K. E. Batcher, "Bitonic sorting on Benes/spl caron/ networks," *Proceedings of International Conference on Parallel Processing*, Honolulu, HI, USA, 1996, pp. 749-753, doi: 10.1109/IPPS.1996.508142.
- [5] M. Marcellino, D. W. Pratama, S. S. Suntiarko and K. Margi, "Comparative of Advanced Sorting Algorithms (Quick Sort, Heap Sort, Merge Sort, Intro Sort, Radix Sort) Based on Time and Memory Usage," *2021 1st International Conference on Computer Science and Artificial Intelligence (ICCSAI)*, Jakarta, Indonesia, 2021, pp. 154-160, doi: 10.1109/ICCSAI53272.2021.9609715.
- [6] Kartik, "Bitonic Sort.", <https://www.geeksforgeeks.org/dsa/bitonic-sort/> (accessed Apr. 22, 2026)